Promo Finder: the most sensational 20-60 s promo clips in a microdrama

Kuku AI Generalist Assignment, Problem 1 (Find the Promos) · Shrawon · November 2026

1. Why we picked Problem 1 (Find the Promos)

For a microdrama app, the promo is the front door. Most new viewers find a show through a 20-60 s clip on Instagram, TikTok or YouTube, and they decide in the first 3 seconds whether to stop scrolling. So the right promo moment directly decides how many people start a show. Today editors find these moments by watching every episode by hand, and Kuku releases new shows every week, so this does not scale. We picked Problem 1 because it is a real, repeated cost for the business, because a tool that reads the screenplays can save hours per show, and because the output (exact start and end times) is concrete and can be checked, so we can measure honestly where the tool is right and where it fails.

2. What it does

It reads the episode screenplays and returns the **top 3 windows of 20-60 seconds in every episode** (and the top 3 of the whole show), preferably about 40 s long: the stretches a scrolling viewer is most likely to stop for. Each window has an exact start, end and duration.

3. Method: sensationality grading

- **Read (code).** Each script is split into timed blocks: dialogue, action, camera and sound, each with its start and end time from the script.
- **Grade every moment (trained model).** Each block gets a **sensationality score** from 0 to 10: how strongly the moment would stop a viewer. Sub-scores (0-3) explain why: conflict, shock, emotion, physicality (slaps, shoves, blood), desire, status (humiliation, power reversal) and open loop (an unanswered question). Heated moments are tagged: slap, scream, threat, reveal...
- **Heat curve (code).** Scores become a heat line over time in half-second steps. Each step takes the hottest block playing then, so a burst of quick camera cuts does not count more than one strong line.
- **Window search (code).** Every window that starts and ends where nobody is mid-sentence and lasts 20-60 s gets S = (0.45 x average heat + 0.20 x peak + 0.20 x first 3 s + 0.15 x last 5 s) x (0.85 + 0.15 x P), where P = 1 - ((L - 40) / 20)^2 rewards lengths L near 40 s.
- **Final assessment (LLM).** Gemini reads each episode's 6 best candidate windows as whole clips, grades each 0-10 and may tighten its start and end. Code snaps those to clean cuts and keeps them within 20-60 s. Final score = 0.6 x Gemini's grade + 0.4 x S. The top 3 that do not overlap are kept.

4. How the grading model was trained (Kaggle)

- **Data:** about 18,000 lines from **MovieSum**, 2,200 film screenplays with dialogue and action marked separately (huggingface.co/datasets/rohitsaxena/MovieSum; Saxena and Keller, ACL Findings 2024), plus all 3,727 blocks of Kuku's 50 episodes.
- **Teacher:** Gemini labelled every line with the rubric above (overall 0-10, seven sub-scores 0-3) at temperature 0. About 3% of lines were labelled twice to measure how consistent Gemini is with itself.
- **Student:** DeBERTa-v3-base (184M parameters) with 8 regression outputs, trained on a free Kaggle T4 GPU in about 10-15 minutes: 4 epochs, batch 32, max 128 tokens, learning rate 2e-5 (head 1e-4), 6% warm-up, weight decay 0.01. Targets are scaled to 0-1 (overall/10, sub-scores/3). Loss: weighted mean squared error, sum over k of $w_k(f(x)_k - y_k)^2$, with the overall score weighted 3 and each sub-score 1.
- **Check:** rank agreement (Spearman) with Gemini on films and Kuku episodes held out entirely (8% of films, every 5th episode); the best checkpoint is kept. Unzipped into the `model/` folder, it grades an episode on a laptop in seconds.

5. Why this method, and what we rejected

- **Rejected: Gemini grading every line at run time.** Hundreds of calls per show are slow and use up the free quota. The trained model does the bulk grading in seconds; Gemini is called once per episode, only for the final call.
- **Rejected: asking the LLM to write timestamps freely.** Models invent or misread times. Here the LLM only picks among real blocks; code sets every time, so no cut ever lands mid-sentence.
- **Fallbacks:** without the model, Gemini grades the blocks itself; without a key, a keyword grader does, which is also the yardstick the model must beat.

6. Test data

- **Kuku's 50 real episodes** (102.7 min). Answer key: independent AI annotators marked the best hook, cliffhanger and must-use clips per episode; a second annotator agreed on 10 of 10 cliffhangers and 8 of 10 hooks.
- **160 extra episodes in 5 new shows, which I synthesized with Gemini** to test the tool on stories it had never seen; each has two marked answer windows.

7. Results

Top 3 windows of the show (all 150 per-episode windows are in `outputs/Top3_windows_per_episode_50_episodes.csv`):

#	Episode	Start	End	Duration	Heated moments	Opens on
1	29	00:37.30	01:03.80	26.5 s	scream, threat	"The magistrate will call it witchcraft."
2	28	00:12.60	00:52.20	39.6 s	collapse, scream, grab	"She looks exactly like a poisoner."
3	21	00:03.50	00:40.40	36.9 s	humiliation, grab, threat	Barn doors slam open: "Toby is dying!"

Check (top 3 windows per episode)	Tool	3 random windows
Kuku: a window starts on the answer key's best hook (within 3 s)	0.56	0.27
Kuku: answer key's must-use clips found (half or more covered)	0.88	0.69
Synthetic shows: marked answer windows matched (overlap 50%+)	0.64	0.35

... mid-episode / end-of-episode answer windows	0.93 / 0.36	-
Cuts inside a spoken line; windows outside 20-60 s	0 / 0	-

Window durations: 20.4-50.4 s, median 33.5 s. These numbers come from the keyword grader, the version that runs here without the model or a key; the same commands use the trained model and Gemini's final assessment once `model/` and the key are in place.

8. My own judgement of the picks

- The top 3 are real confrontations (a witchcraft accusation, a poisoning charge, a dying child) that open on a strong line: usable promos, though not the show's most iconic moments.
- Line-by-line grading over-rates loud lines ("I don't know!") and misreads words: "don't look so wounded" is tagged as a wound. It is strong on mid-episode clashes (0.93) and weak on end-of-episode turns (0.36), where the words are calm but the situation is tense. Gemini's whole-clip assessment is there to catch exactly this.

9. Where it fails

- **Text only:** it cannot see acting, faces or music; a strong line on paper can play flat on screen.
- **One line at a time:** the grader cannot see setup or payoff, so a window can need context a stranger lacks, or end after the tension resolves.
- **AI taste, not audience data:** scores copy Gemini's idea of "sensational", not what viewers really watch. Section 10 explains how real viewer data fixes this.
- **Ad safety:** slaps, blood and threats score highest, and ad platforms may restrict exactly those clips.
- **Licence:** MovieSum is for non-commercial use only; a production model needs licensed or in-house scripts.

10. The biggest improvement: learning from what viewers actually watch

Right now our promo finder model learns from what an AI thinks is sensational. It has never seen how real viewers react. Our promo finder model can make severe improvements after collecting data of the most rewatched and most watched moments of the original episodes, after they are launched and 100+ users have watched them. This is real human response, so it is the best label we can get.

- **What we collect (per episode, per second):** how many viewers are still watching (retention), how many seek back to replay a moment (rewatch), where viewers drop off, and where they share or take a screenshot. We wait until at least 100 users have watched an episode, so a few viewers cannot swing the result. Only totals are stored, never data about any single user.
- **How we turn it into a score:** every episode loses viewers over time anyway, so we compare each second with the normal drop for that point in an episode. A moment that keeps more viewers than expected, or that many users rewatch, gets a high **audience score** (0-10). Each screenplay block takes the audience score of the seconds it covers.
- **How we join it with the Kaggle-trained model:** we do not start again. We take the model we already trained on Kaggle and add one more output, the audience score, next to its 8 existing outputs. We fine-tune it from the saved checkpoint on the launched episodes, with a low learning rate (1e-5) for 2-3 epochs, and keep the old Gemini labels in the loss so it does not forget what it already learned. The model now learns both what reads as sensational and what viewers really stop for.
- **How the promo finder uses it:** for an episode that is already launched, the heat curve mixes the model's score with the real audience curve (for example 50/50), so promos come straight from the most-watched moments. For a new show that nobody has watched yet, the fine-tuned model predicts the audience score from the script alone. So every launched show makes the promos of the next new show better.
- **Closing the loop:** we also track how each posted promo performs (3-second hold rate, watch time, clicks to the show) and use it to tune the weights of the window formula. We retrain every month or after every new batch of shows, and A/B test the new model against the old one before switching.
- **Where it will fail in this case: spoilers.** The most rewatched moments are often the main twist of the plot or the key secrets of the show, for example the moment the hero is revealed to be a Duke. A promo built on that moment gives the twist away, and viewers have less reason to watch the show. The tool can flag windows that contain a reveal or come from late in the series, but it cannot know which secrets the story needs to keep. **So before any promo is released, a human should always watch it, understand it and review it.**
- **Other limits of watch data:** a rewatch can mean a scene was confusing, not exciting. Early episodes always have more viewers than later ones. And the data only exists after launch, so a brand-new show still depends on the trained model alone.

11. Human in the loop

- An editor reviews 3 windows per episode in the app instead of watching everything: minutes per show, not hours.
- They check what the tool cannot: does it make sense to a stranger, does it end unresolved, is it safe for ads; they can nudge a cut by a beat. Every approve or reject is saved and becomes training data.

12. Running it, and AI use

- **Gemini key:** create one at aistudio.google.com/apikey and put it in the file `.env` in the project folder, one line: `GEMINI_API_KEY=your-key`. Start with `python app.py`.
- **AI tools:** Claude wrote and tested the code and the answer key; Gemini synthesized the test shows, labelled the training data and does the final assessment. **Where AI got it wrong:** Claude's first duplicate filter treated different episodes as the same story because the synthetic shows reuse stock lines; testing on them caught it.